

Blocking I/O

Blocking I/O means a thread pauses execution and waits until an input/output operation completes.

During that waiting time, the thread cannot do other useful work.

This is a major limitation in scalable server design.

1. Core Idea

When a client sends a request:

None

Client connects to server

Server thread calls `read()`

If data is not ready → thread waits

That thread remains blocked until data becomes available.

2. How It Works Internally

In operating systems, a server thread may call:

None

`read(fd, buffer)`

Where:

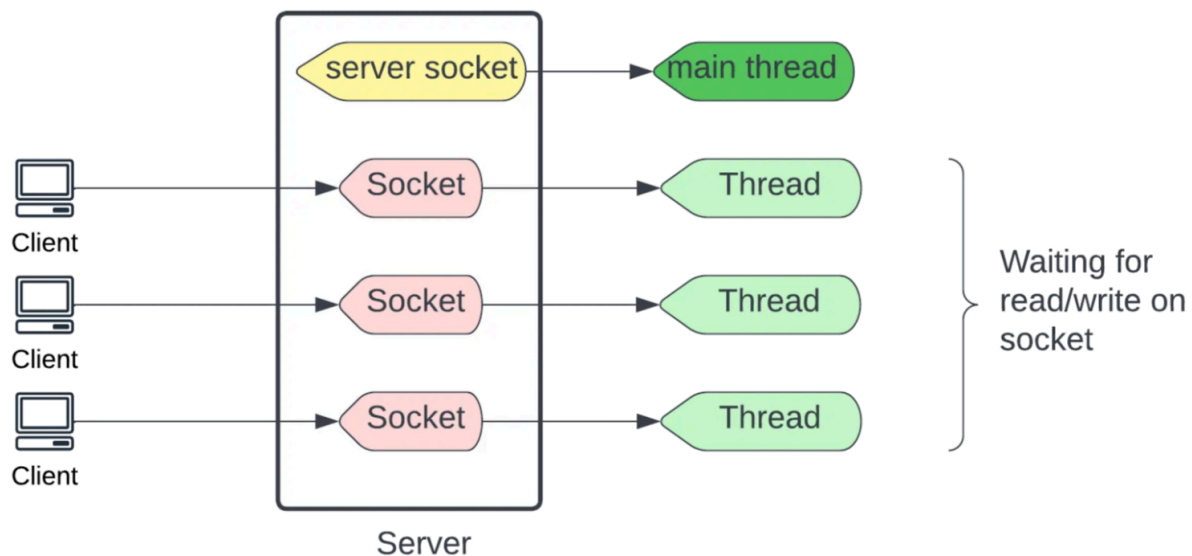
- **fd** = socket file descriptor
- **buffer** = memory buffer for incoming data

The kernel copies data from network socket into memory buffer.

Until enough data arrives:

None

Thread sleeps / blocks



3. What Happens During Blocking

Example:

None

Client connected but sends data slowly

Thread waits seconds for payload

During that time:

- Cannot serve another request
- Holds thread resources
- Uses memory stack
- Scheduler still tracks it

4. Network Buffer Concept

Incoming network data is first stored in kernel/network buffers.

None

NIC → Kernel Buffer → Application Buffer

Application thread waits until readable data is available.

5. Why Blocking I/O Is a Problem

Wastes Threads

Thousands of idle connections may mean thousands of waiting threads.

Poor CPU Utilization

CPU can't progress that blocked request.

Memory Cost

Each blocked thread still consumes memory.

Limits Concurrency

None

More users = more threads = less efficiency

6. Example Scenario

Suppose:

None

10,000 connected clients

9,500 are idle

500 active

With blocking model:

None

10,000 threads may exist

Most are waiting

Very inefficient.

7. Why One Thread per Connection Fails

Because blocking I/O ties:

None

1 waiting connection = 1 occupied thread

At large scale, this does not scale well.

8. Better Alternatives

Modern servers use:

- Non-blocking I/O
- Event loops
- I/O multiplexing (`epoll`, `kqueue`)
- Async runtimes

Used by:

- Redis
- NGINX
- Node.js

9. Interview Insight

If asked why non-blocking servers scale better:

None

Because threads do not wait per connection;
one thread can manage many sockets efficiently.

10. Mental Model

None

Blocking I/O = Stop working until data arrives

One-Line Summary

Blocking I/O pauses a thread until network or disk data becomes available, making thread-per-connection servers inefficient and hard to scale for large numbers of concurrent clients.